

Modified TCP NewReno for Wireless Networks

Ahmed Khurshid

Department of Computer Science
University of Illinois at Urbana-Champaign
Champaign, IL, United States
Email: khurshid@illinois.edu

Md. Humayun Kabir¹, and Rajkumar Das²

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology
Email¹: mfkabir@cse.buet.ac.bd
Email²: rajkumardas@cse.buet.ac.bd

Abstract—Transmission Control Protocol (TCP) congestion control algorithm works well for the **wired networks** where most of the **timeouts and 3-dupacks are the result of congestion**. In the wireless networks, where random segment loss due to bit errors is a dominant concern, the arrival of duplicate acknowledgements or even the retransmission timeouts do not necessarily denote congestion. In those cases, throttling transmission rate is not necessary rather detrimental. For this reason, existing TCP congestion control algorithms fail to perform well in wireless networks. In this paper, we propose a new TCP congestion control algorithm TCP NewReno+ that performs better in wireless networks. We tested our proposed algorithm using the network simulator (ns-2) and it shows better performance than many other TCP variants in the wired, wireless and the mixed networks.

I. INTRODUCTION

Transmission Control Protocol (TCP) [1] is the principal transport protocol used in the Internet. TCP ensures a reliable, ordered, connection oriented, byte streamed full duplex communication. It was originally designed to provide reliable data delivery over the wired networks for a **limited range of transmission rates and propagation delays**. It performs both flow control and congestion control. The purpose of flow control is not to **overwhelm the receiver of a TCP connection**. During TCP communication, the TCP receiver constantly informs the TCP sender about its current buffer capacity and the TCP sender tunes its transmission rate accordingly. On the other hand, congestion control is a network wide issue. Its purpose is to control the transmission rate so that the sender does not transmit in excess of the capacity of the network. One of the strengths of TCP lies in its congestion control mechanism proposed in the cornerstone work by Van Jacobson [2]. Generally, the congestion information is not advertised by the congested nodes. The sending entity adjusts a **congestion window based on successful transmissions and uses the congestion window as the maximum limit to transmit data**. Setting the congestion window too small might result in **under utilization of network resources**. On the other hand, a large congestion window may **over feed the network** that might result in dropping of **segments at the congested nodes**.

The congestion control algorithm used in the TCP/IP protocol suite [2], [3] is a sliding window mechanism that uses segment loss to detect congestion.

The TCP sender probes the network state by gradually increasing the window of segments that are outstanding in the network until the network becomes congested and drops segments. **Initially, the increase is exponential and this phase is called "Slow-start"**. This phase is intended to quickly grab the available bandwidth. When the window size reaches a **slow-start threshold (called ssthresh)**, TCP enters into the second phase called **"Congestion Avoidance"**, where the increase becomes **linear**. This is done to make the TCP sender less aggressive in probing for the available bandwidth. Clearly, it is desirable to set the threshold to a value that approximates the connection's "fair share". The optimal value for the slow-start threshold is the one that corresponds to the number of segments in flight in a pipe when TCP transmission rate is equal to the available bandwidth [4], i.e. when its transmission window is equal to the available bandwidth-delay product.

The current strategy taken by TCP in controlling network congestion is not adequate to perform well in wireless networks. When a loss is detected either through duplicate acknowledgements, or through the expiration of the retransmission timer, the connection backs off by shrinking its congestion window. If the loss is indicated by the three-duplicate-acknowledgement event, TCP Reno [3], one of the variants of the original TCP algorithm, attempts to perform a **"fast recovery"** by **retransmitting the lost segment and halving the congestion window**. If the loss detected through a retransmission timeout, the congestion **window is reset to 1**. In either case, when the congestion window is **reset**, TCP's window-based probing needs several round-trip times to restore its value to the near-capacity. This problem is exacerbated when random or sporadic losses occur. Random losses are losses not caused by congestion at the bottleneck link. They are common in the wireless channels. In this case, a burst of lost segments is erroneously interpreted by a TCP source as an indication of congestion, and dealt with by shrinking the **sender's window**. Such action, clearly, does not alleviate the random loss condition and it merely results in reduced throughput. The larger the bandwidth-delay product, the larger the performance degradation caused by such action.

For this reason, the congestion control strategy employed by TCP works fine in wired networks, where most of the timeouts and the delivery of out of order segments are caused by network congestion. However, in wireless networks, a good percentage of timeouts and **reception of out of order segments**

happen due to the bit error rather than congestion. In those cases, throttling transmission rate does not help as this action results in under utilization of network bandwidth without any improvement in network activities. As the existing TCP congestion control algorithms cannot distinguish between congestion and bit error timeouts, it fails to give good performance in wireless networks. In this paper, we propose a new TCP congestion control algorithm in order to get better performance in wireless networks. In particular, our algorithm refines the multiplicative decrease algorithm of TCP NewReno. We are using some statistical counters to track the frequencies of the occurrences of timeouts and 3-dupacks. Different ratios of these counter values are then used to differentiate a congestion event from a non-congestion event. We are also tracking the time difference between two consecutive timeouts to figure out whether timeouts are caused by network congestion or random bit errors.

The rest of the paper is organized as follows. Related research works done by other researchers are discussed in Section II. Our proposed new TCP congestion control algorithm NewReno+ is presented in Section III. Detailed performance analysis of the proposed algorithm with the help of Network Simulator version 2 (ns-2) [5] is presented in Section IV. Section V concludes the paper with some hints on the future research in this direction.

II. RELATED RESEARCH WORKS

In [6] Subramanian et al. proposed an enhancement to TCP called LT-TCP (Loss Tolerant TCP) that performs well in extreme wireless environments. They showed that after certain bit error rate, where the original TCP fails miserably, LT-TCP shows good performance. Their work relies on ECN (Explicit Congestion Notification) from the network routers. LT-TCP uses FEC (Forward Error Correction) mechanism to recover segment errors and losses. Based on the current network condition, a certain amount of error correcting segments are pre-generated and kept in the transmission queue. Some of these are called PFEC (Proactive Forward Error Correction) segments and are sent along with the new data segments. The rest are called RFEC (Reactive Forward Error Correction) segments and are sent during retransmission of previously sent segments. LT-TCP also adaptively manages the maximum segment size to ensure a certain minimum number of FEC segments in the transmission window. Using slots from the transmission window for additional error correcting segments, results in reduced number of slots for the actual data. This reduces the throughput of a TCP connection. Moreover, generation of error correcting segments at the sender consumes processor time. Similarly, the use of error correcting segments at the receiving end in order to recover damaged segments also needs some extra processing activities.

Indirect-TCP (I-TCP) [7], uses TCP splitting approach with different flow control and congestion control mechanism on the wireless and wired parts of a network. The TCP connection is splitted into two connections at the point where the wired and wireless sub networks meet, i.e. at the base station. The

base station keeps one TCP connection with the fixed host, while it uses another connection with a protocol specially designed for better performance over wireless links for the mobile host. The base station acknowledges the segments as soon as it receives them. An acknowledgement can arrive at the sender even before the corresponding segment has been received by the receiver. The base station forwards the segments and buffers a segment until it receives the acknowledgement from the mobile host. A base station transparently transfers state information to another base station during handoffs. I-TCP provides faster adaptation to mobility and wireless link breaks. However, I-TCP has larger processing overhead and it violates the end-to-end semantics of a TCP connection.

In [8] Tsaoussidis et al. proposed an end-to-end proposal called TCP Probing. In this scheme when a data segment is delayed or lost, the sender enters into a probe cycle instead of retransmitting and reducing the congestion window size. In a probe cycle only the probe segments are exchanged between the sender and receiver to monitor the network and the regular transmission is suspended. The probes are TCP segments with header option extensions and no payload. A probe cycle terminates when the sender can make two successive RTT measurements with the aid of receiver probes. In case of persistent error, TCP decreases its congestion window (cwnd) and ssthresh. For transient random error, the sender resumes transmission at the same window size that it was using before entering into the probe cycle. Although the probe segments are small, they increase the network load even when the network is highly congested.

TCP Westwood (TCPW) [9][10] is a modified version of TCP Reno. TCPW enhances the window control and back off process. Here, a TCP sender performs an end-to-end estimate of the bandwidth available along the connection by measuring the rate of returning acknowledgements and the amount of bytes delivered to the receiver during a certain interval. Whenever a sender perceives a segment loss (i.e. a timeout occurs or 3-dupacks are received), the sender uses the bandwidth estimate to properly set the congestion window (cwnd) and the slow-start threshold (ssthresh). By backing off the cwnd and the ssthresh to the values those are based on the estimated bandwidth rather than simply halving the current values as TCP Reno does, TCP Westwood avoids overly conservative reductions of cwnd and ssthresh; and thus it ensures a faster recovery. Although TCPW shows better performance than other TCP variants in contention free wireless networks (e.g. a dedicated channel between a VSAT and a satellite), it fails miserably when deployed in multi-access wireless networks where multiple wireless nodes share the same radio frequency using collision avoidance type of scheme. TCPW relies on a consistent flow of acknowledgements from the receiver in order to calculate a near-to-actual estimate of the available network bandwidth. Whenever the acknowledgement stream is disrupted, TCPW's estimation process gives wrong results and degrades the overall performance of a TCP connection.

The receipt of four back-to-back identical ACKs during congestion avoidance is referred to as triple duplicate ACKs,

which causes TCP Reno [3] sender to perform a fast retransmit and to enter into fast recovery phase. In fast retransmit, the sender retransmits the lost segment, and sets the slow start threshold (sssthresh) to $cwnd/2$ and congestion window to new sssthresh plus 3. In fast recovery, the sender continues to increase the congestion window by one for each subsequent duplicate ACK received since this reception indicates that some segments are still reaching the destination and thus can be used to trigger new segment transmissions. The sender is thus allowed to transmit new segments abiding by the limit imposed by its new congestion window. However, the receipt of a non-duplicate ACK in TCP Reno results in window deflation. Congestion window is set to current sssthresh, i.e., the congestion window size in effect when the sender entered into fast recovery. Fast recovery phase terminates and normal congestion avoidance behavior resumes. Therefore, when multiple segments are dropped from the same window TCP Reno enters into and leaves from fast recovery phase multiple times, which again causes multiple reductions of the congestion window. TCP NewReno [11] modified TCP Renos fast recovery algorithm to avoid multiple reductions of the congestion window. TCP NewRenos slow start, congestion avoidance and fast retransmit phases are identical to that of TCP Reno. TCP NewReno defines a full ACK and a partial ACK as follows. A full ACK acknowledges all the segments that were outstanding at the start of fast recovery. A partial ACK acknowledges some but not all of this outstanding segments. TCP Reno terminates fast recovery phase in the event of a partial ACK. This action leads to multiple entrances and leavings of fast recovery phase due to multiple segment losses in the same window. For this reason, upon the receipt of a partial ACK TCP NewReno retransmits the segment next in sequence and reduces the congestion window by one less than the number of segments acknowledged by the partial ACK. TCP NewReno remains in the fast recovery phase until a full ACK is received. On the receipt of a full ACK, TCP NewReno sets $cwnd$ to sssthresh, terminates fast recovery phase, and resumes congestion avoidance phase. TCP NewReno recovers from multiple segment losses in the same window by retransmitting one lost segment per RTT.

III. PROPOSED TCP CONGESTION CONTROL ALGORITHM

In order to control congestion TCP throttles its transmission rate so that the overloaded routers do not get flooded with the new segments and get some time to drain out their queues without dropping the segments. This action helps the network to ease the congestion if there is real congestion in the network. However, if segments are lost due to bit error then there is no gain in reducing the transmission rate. In this case, the sender should continue transmitting at the original rate and try to deliver as many segments as possible in the midst of random bit errors. Throttling transmission rate will not do any good in this scenario. Hence, some new strategies should be introduced in TCP such that it can detect segment loss due to bit error and act accordingly.

We keep a running count of the number of timeouts and the number of 3-dupacks experienced during an interval. Whenever the sender experiences a timeout or 3-dupack event, we compute the ratio of the number of timeouts to the number of 3-dupacks. Our observation shows that a very small ratio is an indication of a bit error event, not a congestion event and a high ratio is more likely an indication of a congestion event.

If the network is congested then the timeout will happen successively. In this case, the time difference between two consecutive timeout events will be roughly equal to the timeout interval of the retransmission timer at that instant. On the other hand, when the network is not congested, the TCP sender will only encounter timeout events whenever the $cwnd$ crosses the current network capacity. That will be quickly resolved by the TCP sender by entering into the slow-start phase. Again, if there are random bit errors, then some segments will be damaged and will be rejected by the receiver. However, duplicate acknowledgements will be returned from the receiver to the source for subsequent segments which are not lost. This will prevent the source from having timeout events and will also enable the source to solve potential loss of segments using fast retransmit and fast recovery. So, in non-congestion scenario the timeouts will be sparse and the time difference between two successive timeouts will be much greater than the retransmission timer's estimated timeout interval at that moment. This observation can also be used during a timeout or 3-dupack event to detect whether the event is a result of real network congestion or due to random segment losses due to bit error. Let, two consecutive timeout occurs at time tx and $tx+1$, and the time difference between these two events is td (i.e. $td = tx+1 - tx$). Also let ti denotes the current estimate of retransmission timer's timeout interval. During a timeout or 3-dupack event, we propose to compute the ratio of ti to td . If the ratio is very small (in between 0.01 to 0.1), our observation shows that this event has been caused by a bit error event, not by the congestion. If the ratio is high (e.g. greater than 0.25) then the event is more likely the result of segment drops in intermediate routers due to congestion. When the possibility of congestion is less but the possibility of bit error is high, our TCP NewReno+ algorithm makes TCP less conservative, i.e. no or little reduction in transmission rate. In 3-dupack-event case, the amount of throttling is not that high due to TCP's fast retransmission and fast recovery strategy. We do not reduce the $cwnd$ and sssthresh that much during a possible bit error event so that TCP can continue with a transmission rate close to the previous rate.

Table 1 lists the parameters used in our TCP NewReno+ algorithm to work along with existing TCP NewReno algorithm.

A. Operations of Procedure "RECEIVE"

At TCP source procedure "RECEIVE" is invoked by the network layer whenever the source receives an acknowledgement segment from the TCP receiver. In this procedure the TCP source performs different actions based on the type of acknowledgement received. In fact, we have kept the original

Table 1: Parameters used in NewReno+

Parameters	Purpose
TC	Count of timeouts
DC	Count of 3-dupacks
TDR	TC : DC
TDR_T	Threshold of TDR
TDR_A	Aging factor of TDR
TI	Current timeout interval of the retransmission timer
TD	Time difference between two successive timeouts
ITR	TI : TD
ITR_T	Threshold of ITR
ITR_A	Aging factor of ITR
Latest_Timeout	Holds the time of occurrence of the last timeout event
cwnd	Latest cwnd
ssthresh	Latest ssthresh
NR_cwnd	cwnd set by TCP NewReno
NR_ssthresh	ssthresh set by TCP NewReno
K_cwnd	cwnd set by NewReno+
K_ssthresh	ssthresh set by NewReno+

implementation of "RECEIVE" specified in TCP NewReno [7] intact.

B. Operations of Procedure "DUPACK_ACTION"

The procedure "DUPACK_ACTION" is invoked whenever the TCP sender gets **three consecutive duplicate acknowledgments (3-dupacks)**. The pseudo code of this procedure is given in Algorithm 1.

Algorithm 1: DUPACK_ACTION

```

DC = DC + 1
TDR = (TDR_A * TDR) + (1 - TDR_A) * (TC / DC)
TD = Current_Time - Latest_Timeout
ITR = (ITR_A * ITR) + (1 - ITR_A) * (TI / TD)
Reset the retransmission timer
Retransmit the presumed lost segment using fast retransmission
CALL SLOWDOWN_ACTION (DUPACK)

```

C. Operations of Procedure "TIMEOUT_ACTION"

This procedure is invoked whenever the retransmission timer expires at the TCP sender. The pseudo code of this procedure is given in Algorithm 2.

Algorithm 2: TIMEOUT_ACTION

```

TC = TC + 1
TDR = (TDR_A * TDR) + (1 - TDR_A) * (TC / DC)
DC = MAX(DC / 4, 1)
Previous_Timeout = Latest_Timeout
Latest_Timeout = Current_Time
TD = Latest_Timeout - Previous_Timeout
ITR = (ITR_A * ITR) + (1 - ITR_A) * (TI / TD)
Reset the retransmission timer
CALL SLOWDOWN_ACTION (TIMEOUT)
Retransmit the segment

```

D. Operations of Procedure "SLOWDOWN_ACTION"

The "SLOWDOWN_ACTION" procedure is the place where the actual throttling actions of the TCP sender take place. It is invoked by both "DUPACK_ACTION" and "TIMEOUT_ACTION" procedures. The pseudo code of this procedure is given in Algorithm 3.

Algorithm 3: SLOWDOWN_ACTION (EVENT_TYPE: Event)

```

IF Event = DUPACK THEN
    Enter into fast retransmission
    Enter into fast recovery
    NR_ssthresh = cwnd / 2
    NR_cwnd = NR_ssthresh + 3
ELSE IF Event = TIMEOUT THEN
    NR_ssthresh = cwnd / 2
    NR_cwnd = 1
END IF
IF TDR < TDR_T AND ITR < ITR_T THEN
    K_ssthresh = 3/4 cwnd
    K_cwnd = 1/2 cwnd
ELSE IF TDR < TDR_T OR ITR < ITR_T THEN
    K_ssthresh = 2/3 cwnd
    K_cwnd = NR_cwnd
ELSE
    K_ssthresh = NR_ssthresh
    K_cwnd = NR_cwnd
END IF
ssthresh = MAX(K_ssthresh, NR_ssthresh)
cwnd = MAX(K_cwnd, NR_cwnd)

```

IV. PERFORMANCE EVALUATION OF TCP NEWRENO+

We used ns-2 simulation software to test the performance of our proposed congestion control algorithm and to compare the same with the other popular TCP variants, such as NewReno and Westwood. We used a mixed network, a wired network with a wireless extension, in this regard. Therefore, the network topology in our simulations has both wired and wireless nodes. **We used 3 wired and 5 wireless nodes.** An access point acts as a gateway between the wired and wireless part of the network. All the nodes in the wired networks have **5 mbps** dedicated link to a head node. The gateway node is connected to the head node using either **a 7 mbps or a 12 mbps link**. This link acts as the bottleneck link in our simulation. We have generated following TCP and UDP connections in different simulation scenarios. One TCP connection between the first wired node (sender) and the first wireless node (receiver). A second TCP connection between the second wired node (sender) and the second wireless node (receiver). One UDP connection between the third wired node (sender) and the third wireless node (receiver). A second UDP connection between the fourth wireless node (sender) and the fifth wireless node (receiver).

UDP connections are generating constant bit rate traffics which are used to create congestion in the network. In order to evaluate the performance of different TCP implementations we have used the number of **unique segments transmitted** by the sender as the comparison parameter. This number represents the throughput of a connection and if it is larger in one TCP implementation than that is in another TCP implementation, the former denotes the superiority over the later. We ran the simulation for **TCP NewReno, TCP Westwood and TCP NewReno+.** All the simulations ran for 250 seconds. When a single TCP connection performance is measured, we had used only one TCP connection between a wired node (sender) and a wireless node (receiver). We have used this scenario to analyze how TCP NewReno+ behaves when it does not have to

compete with other concurrent TCP connections. During this test we set the bottleneck link bandwidth to 7 mbps. In order to analyze the performance of TCP NewReno+ in a multi-connection scenario we ran the simulations using both TCP connections; one TCP connection between the first wired node (sender) and the first wireless node (receiver) and the second TCP connection between the second wired node (sender) and the second wireless node (receiver). In this scenario, the bottleneck link bandwidth was set to 12 mbps. In both cases, we kept the bottleneck bandwidth slightly higher than that is generally required by the TCP connection(s) so that we can introduce different levels of congestion by controlling the data rate of background UDP traffics.

A. Simulation Results and Analysis

Table 2 shows simulation results obtained after running a single TCP connection using different TCP variants and TCP NewReno+.

Table 2: Performance with single TCP connection

Error Rate(%)	NewReno	Westwood	NewReno+
1	15481	15449	15531
2.5	15207	15002	15275
5	14290	13870	14418

In the above simulation runs all the UDP traffics were active. From the simulation results it is clearly evident that NewReno+ outperforms all other TCP variants, even TCP Westwood, which was specially designed for wireless networks. TCP Westwood relies on consistent supply of acknowledgement segments from the receiver to estimate the available bandwidth of the network. TCP Westwood's performance depends highly on the precision of the above estimation. In presence of bit errors, the wireless nodes fail to transmit acknowledgement segments successfully to their respective peers. So the multi-access nature of a wireless network and the presence of random bit errors refrain TCP Westwood from having an accurate estimation of available network bandwidth.

The performance improvement of NewReno+ can be attributed to its less conservative reaction during segment losses due to random bit errors. Whenever NewReno+ detects a possible non-congestion event it does not reduce its transmission rate too much. So it continues transmitting at a good rate and can deliver more segments in the midst of wireless bit errors. But as other TCP variants (Tahoe, Reno and NewReno) drastically reduces the congestion window whenever a segment loss is detected they fail to achieve a good throughput. In case of NewReno, fast retransmission and fast recovery are capable of ensuring a good throughput when multiple segments are dropped from the same window. However, if segment drops are sporadic in nature, consecutive reception of 3-dupacks continues the halving of the congestion window even though the segments are dropped due to bit error. NewReno+ detects segment losses due to bit error with high precision and keeps a steady flow of segments towards the destination to ensure a good throughput. Again in real congestion, NewReno+ does not behave aggressively and hence do not worsen the

congestion in the network. This behavior is very significant where two concurrent TCP connections are used.

Table 3 shows the simulation results obtained from running two concurrent and homogeneous TCP connections. Here, the average number of unique segments sent by the two connections has been recorded. UDP traffics were present during these simulation runs.

Table 3: Performance with two TCP connections

Error Rate(%)	NewReno	Westwood	NewReno+
1	7817	7820	7823
2.5	7702	7704	7712
5	7475	7479	7497

From the above data it is clearly evident that NewReno+ is able to inject more unique segments into the network than both TCP NewReno and TCP Westwood. These observations confirm that TCP NewReno+ does not affect the operation of concurrent TCP connections. If TCP NewReno+ were too much aggressive then it would have adversely affected the other TCP connections, who are sharing the same bottleneck link. If a TCP connection mistakenly remains aggressive during network overload time, a large number of segments will be dropped at the congested node. These dropped segments will consist of segments from the aggressive connection and also segments from the other moderate TCP connections. So all the moderate connections will experience more timeout events and hence will throttle their transmission rate. Moreover, the aggressive connection will also experience more timeout events that will drastically reduce its rate of transmission. So in the long run the average throughput of the overall network will be low. This situation will continue each time a TCP connection shows aggressive behavior during real network congestion. TCP NewReno+ does not reduce cwnd and/or ssthresh too much until it is ensured that the timeout or 3-dupack event has occurred due to a congestion related event. So the actions TCP NewReno+ takes during false alarm of network congestion do not produce any burden on the network and concurrent TCP connections. All the simulation results presented above have also shown that the gain of our TCP NewReno+ increases with the increase in the error rate in the network, which is a desired property of a TCP algorithm that is designed to overcome the bit error problem.

V. CONCLUSION

The congestion control algorithm proposed in this paper can be incorporated with any existing TCP variant and is capable of performing well in heterogeneous networks (e.g. wired-cum-wireless network). It is end-to-end in nature and modifies only the sender-side TCP implementation. It keeps the TCP receiver and the network unaware of the modifications. This feature makes it suitable for deploying in real life scenario and does not impose any burden on the internal network. Changes in the estimated round-trip time (RTT) of a TCP connection sheds some light on the current network load. By observing the change pattern of RTT, a TCP source can deduce the optimum level of throughput that will enable the

source to utilize the available bandwidth successfully without overburdening the network. So some type of record keeping of previous RTT values and decisions based on those records can be incorporated in the congestion control algorithm to improve TCP's performance in mixed networks. We will incorporate the change pattern of RTT in congestion control in our future work

REFERENCES

- [1] Postel, J., "Transmission Control Protocol", STD 7, RFC 793, September 1981.
- [2] Jacobson, V., "Congestion Avoidance and Control", Proceedings of SIGCOMM '88, ACM, pp. 314-329, Stanford, CA, August 1988.
- [3] Allman, M., Paxson, V. and Stevens, W., "TCP Congestion Control", RFC 2581, April 1999.
- [4] Hoe, J. C., "Improving the Start-up Behavior of a Congestion Control Scheme for TCP", Proceedings of ACM SIGCOMM '96, pp. 270-280, Stanford, CA, USA, August 1996.
- [5] "The Network Simulator - ns-2", <http://www.isi.edu/nsnam/ns/>.
- [6] Subramanian, V., Kalyanaraman, S. and Ramakrishnan, K. K., "An End-to-End Transport protocol for Extreme Wireless Network Environments", Military Communications Conference, MILCOM 2006, pp. 1-7, 2006.
- [7] Bakre, A. and Badrinath, B. R., "I-TCP: Indirect TCP for Mobile Hosts", Proceedings of the 15th International Conference on Distributed Computing Systems, IEEE, pp. 136-143, May 1995.
- [8] Tsaooussidis, V. and Badr, H., "TCP-probing: Towards an error control schema with energy and throughput performance gains", Proceedings of the 8th IEEE Conference on Network Protocols, Japan, November 2000.
- [9] Mascolo, S., Casetti, C., Gerla, M., Lee, S. S. and Sanadidi, M., "TCP Westwood: congestion control with faster recovery", UCLA CSD Technical Report #200017, 2000.
- [10] Mascolo, S., Casetti, C., Gerla, M., Sanadidi, M., Y. and Wang, R., "TCP Westwood: Bandwidth Estimation for Enhanced Transport over Wireless Links", Proceedings of ACM Mobicom 2001, pp. 287-297, Rome, Italy, July 16-21, 2001.
- [11] Floyd, S., Henderson, T., "The NewReno Modification to TCP's Fast Recovery Algorithm", RFC 2582, April 1999.